

# 文本分类实现实验报告

## 一、实验目的

本次实验旨在理解和掌握不同文本表示方法在文本分类任务中的基本原理与实际效果。具体目标包括：

1. 理解不同文本表示方法的基本思想；
2. 掌握文本分类任务的基本实验流程；
3. 熟悉 Logistic Regression 在文本分类中的使用；
4. 掌握 Word2Vec、GloVe 等词向量的基本使用方法；
5. 熟悉 BERT 类预训练语言模型的 Fine-tuning 流程；
6. 能够使用 Accuracy 和 Macro-F1 对文本分类模型进行评价；
7. 能够对不同文本表示方法的性能进行基本比较和分析。

## 二、数据集

### 1. NYT 新闻数据集

NYT 数据集为本次实验的主要文本分类数据集，包含两列：

- `text`：新闻文本
- `label`：新闻所属类别

数据集共 11519 条样本，类别数为 3。

### 2. AG News 数据集

AG News 数据集提供新闻文本（`text`），共 90000 条。本次实验中，AG News 仅用于训练 Word2Vec 词向量，不参与最终分类评价。

## 三、数据划分

首先对 NYT 数据集进行随机打乱，并统一按以下比例划分：

| 数据集            | 比例  | 样本数  |
|----------------|-----|------|
| Training Set   | 80% | 9215 |
| Validation Set | 10% | 1152 |
| Test Set       | 10% | 1152 |

划分时固定 `random_state=42`，并将划分结果保存为 `split.pkl`，供 Task 1、Task 2、Task 3 共用，以保证不同方法之间具有可比性。

运行 `split_data.py` 的输出如下：

```
D:\anacondaishere\python.exe D:\新建文件夹\自然语言处理\pythonProject1\split_data.py
NYT shape: (11519, 2)
Train: 9215
Val : 1152
Test : 1152
已保存 split.pkl
进程已结束，退出代码为 0
```

## 四、评价指标

本次实验统一使用以下两个指标：

1. **Accuracy**：分类正确的样本数量占全部测试样本数量的比例。
2. **Macro-F1**：分别计算每个类别的 F1 Score，再对所有类别的 F1 取平均。对于类别分布可能不均衡的文本分类任务，Macro-F1 能更全面地反映模型在不同类别上的分类能力。

所有模型最终均在 **NYT Test Set** 上进行评价。

## 五、Task 1：Bag of Words

### 1. 方法说明

本部分使用两种 Bag-of-Words 文本表示方法，分类器统一使用 Logistic Regression。

#### (1) Binary Bag of Words

构建词表 Vocabulary 后，每篇文档表示为一个  $|V|$  维向量。对于词表中的第  $i$  个单词：

- 若该词出现在文档中， $x_i = 1$
- 否则， $x_i = 0$

即只考虑单词是否出现，不考虑出现次数。

#### (2) Word Frequency

仍使用词表中的所有单词构建文档向量，但记录每个单词在当前文档中的出现次数。即  $x_i$  表示词表中第  $i$  个词在当前文档中的出现次数。

### 实现细节

- 分词：`text.lower().split()`
- 词表构建与向量化：`sklearn.feature_extraction.text.CountVectorizer`
- 分类器：`LogisticRegression(max_iter=1000)`

### 2. 实验结果

运行 `task1_bow.py` 的输出如下：

```
D:\anacondaishere\python.exe D:\新建文件夹\自然语言处理\pythonProject1\task1_bow.py
Train: 9215 Val: 1152 Test: 1152

===== Binary BoW =====
Vocabulary size: 227613
Val Accuracy: 0.9852 Macro-F1: 0.9645
Test Accuracy: 0.9913 Macro-F1: 0.9797

===== Word Frequency =====
Vocabulary size: 227613
Val Accuracy: 0.9818 Macro-F1: 0.9591
Test Accuracy: 0.9896 Macro-F1: 0.9771

===== Task 1 结果汇总 =====
      method  val_acc  val_f1  test_acc  test_f1
Binary BoW    0.985243  0.964472  0.991319  0.979717
Word Frequency 0.981771  0.959127  0.989583  0.977111

已保存 task1_results.csv

进程已结束，退出代码为 0
```

结果汇总：

| 方法             | Val Accuracy | Val Macro-F1 | Test Accuracy | Test Macro-F1 |
|----------------|--------------|--------------|---------------|---------------|
| Binary BoW     | 0.9852       | 0.9645       | <b>0.9913</b> | <b>0.9797</b> |
| Word Frequency | 0.9818       | 0.9591       | 0.9896        | 0.9771        |

### 3. 结果分析

1. 两种词袋方法在 NYT Test Set 上都取得了很高的性能，Test Accuracy 均超过 0.98，说明 NYT 数据集的类别区分度较强，词袋表示已经能够捕捉到大部分关键信息。
2. Binary BoW 略优于 Word Frequency。可能的原因是：新闻类别往往由“是否出现某些关键词”决定，而出现次数受文本长度和写作风格影响较大，反而不如二值表示稳定。
3. Macro-F1 略低于 Accuracy，说明少数类别上的分类表现稍弱，这与类别分布不完全均衡有关。
4. 验证集与测试集结果接近，说明模型没有明显过拟合，数据划分合理。

## 六、Task 2: Word2Vec / GloVe

### 1. 方法说明

本部分使用基于 Word Embedding 的方式表示文档。统一要求：

- 100 维词向量
- 分类器：Logistic Regression
- 文档向量：文档内所有有效词向量的平均

对于一篇包含  $n$  个有效单词的文档，假设对应词向量为  $e(w_1), e(w_2), \dots, e(w_n)$ ，则文档向量为：

$$e(d) = (1 / n) * \sum e(w_i)$$

本次共完成三组实验：

### (1) Pre-trained GloVe

使用公开的预训练 GloVe Embedding ( `glove.6B.100d.txt` ) , 选择 100 维词向量作为单词表示。

### (2) Word2Vec Trained on AG News

使用 AG News 的 90000 条文本数据自行训练一个 100 维 Word2Vec 模型, 工具为 `gensim` 。

### (3) Word2Vec Trained on NYT

使用 NYT 的 **训练集** 文本训练 100 维 Word2Vec 模型。为避免数据泄漏, 验证集和测试集不参与训练。

得到词向量后, 对每篇 NYT 文档:

1. 分词;
2. 找到每个词对应的词向量;
3. 对文档内所有有效词向量取平均;
4. 得到 100 维文档向量;
5. 使用 Logistic Regression 完成分类。

## 2. 实验结果

运行 `task2_word2vec.py` 的输出如下:

```
D:\Python12\python.exe D:\新建文件夹\自然语言处理\pythonProject1\task2_word2vec.py
Train: 9215 Val: 1152 Test: 1152

正在加载 GloVe ...
GloVe 词表大小: 400000

正在读取 AG News ...
AG 文本数: 90000
正在训练 Word2Vec on AG News ...
AG Word2Vec 词表大小: 116795

正在训练 Word2Vec on NYT (只用训练集) ...
NYT Word2Vec 词表大小: 227613

===== GloVe (pre-trained) =====
Train token hit rate: 0.8685
Val Accuracy: 0.9766 Macro-F1: 0.9458
Test Accuracy: 0.9852 Macro-F1: 0.9646

===== Word2Vec on AG News =====
Train token hit rate: 0.9287
Val Accuracy: 0.9679 Macro-F1: 0.9273
Test Accuracy: 0.9792 Macro-F1: 0.9485
```

```
===== Word2Vec on AG News =====
Train token hit rate: 0.9287
Val Accuracy: 0.9679 Macro-F1: 0.9273
Test Accuracy: 0.9792 Macro-F1: 0.9485
```

```
===== Word2Vec on NYT =====
Train token hit rate: 1.0000
Val Accuracy: 0.9722 Macro-F1: 0.9373
Test Accuracy: 0.9826 Macro-F1: 0.9556
```

```
===== Task 2 结果汇总 =====
      method  val_acc  val_f1  test_acc  test_f1
GloVe (pre-trained) 0.976562 0.945778 0.985243 0.964617
Word2Vec on AG News 0.967882 0.927343 0.979167 0.948478
      Word2Vec on NYT 0.972222 0.937315 0.982639 0.955615
```

已保存 task2\_results.csv

结果汇总：

| 方法                  | 词表大小   | 命中率    | Val Acc | Val Macro-F1 | Test Acc      | Test Macro-F1 |
|---------------------|--------|--------|---------|--------------|---------------|---------------|
| GloVe (pre-trained) | 400000 | 0.8685 | 0.9766  | 0.9458       | <b>0.9852</b> | <b>0.9646</b> |
| Word2Vec on AG News | 116795 | 0.9287 | 0.9679  | 0.9273       | 0.9792        | 0.9485        |
| Word2Vec on NYT     | 227613 | 1.0000 | 0.9722  | 0.9373       | 0.9826        | 0.9556        |

### 3. 结果分析

1. 三组实验的 Test Accuracy 均在 0.98 左右，Macro-F1 在 0.95 左右，说明词向量平均表示已经能较好地捕捉文档语义。
  2. **GloVe 表现最好**。虽然其训练语料与 NYT 不完全一致，且命中率只有 0.8685，但由于预训练语料规模大、覆盖广，词向量质量较高，因此整体表示效果最好。
  3. **Word2Vec on AG News 略差**。AG News 与 NYT 虽同属新闻领域，但 AG News 文本偏短、偏标题化，与 NYT 正文风格存在差异，导致词向量迁移效果稍弱。
  4. **Word2Vec on NYT 居中**。其词表完全来自 NYT 训练集，命中率达到 1.0000，但由于训练语料仅有 9215 篇，规模较小，词向量质量不如 GloVe。
  5. 与 Task 1 相比，三种词向量方法的性能均低于 Binary BoW。这说明在本数据集上，高维稀疏的词袋表示反而更有效；平均词向量会丢失词序和部分局部信息，导致性能略有下降。
-

## 七、Task 3: Pre-trained Neural Models

### 1. 方法说明

本部分使用预训练语言模型 `bert-base-uncased` 直接完成文本分类任务，进行 Fine-tuning。

训练要求：

- Pre-trained model: `bert-base-uncased`
- Maximum sequence length: 64
- Number of epochs: 3

实现细节：

- Tokenizer: `BertTokenizer.from_pretrained('bert-base-uncased')`
- 模型: `BertForSequenceClassification`
- 训练框架: Hugging Face `Trainer`
- 训练参数:
  - `num_train_epochs=3`
  - `per_device_train_batch_size=16`
  - `learning_rate=2e-5`
  - `eval_strategy='epoch'`
  - `load_best_model_at_end=True`
  - `metric_for_best_model='macro_f1'`

训练完成后，在 NYT Test Set 上评价。

### 2. 实验结果

运行 `task3_bert.py` 的过程及关键输出如下：

```
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

开始训练 BERT ...
6%|  | 100/1728 [04:20<1:07:37, 2.49s/it]{'loss': '0.3531', 'grad_norm': '3.3', 'learning_rate': '1.885e-05', 'epoch': '0.1'}
12%|  | 200/1728 [08:28<1:02:34, 2.46s/it]{'loss': '0.1296', 'grad_norm': '0.5548', 'learning_rate': '1.77e-05', 'epoch': '0.2'}
17%|  | 300/1728 [12:40<59:44, 2.51s/it]{'loss': '0.09421', 'grad_norm': '16', 'learning_rate': '1.654e-05', 'epoch': '0.520'}
23%|  | 400/1728 [16:59<54:42, 2.47s/it]{'loss': '0.1212', 'grad_norm': '10.93', 'learning_rate': '1.538e-05', 'epoch': '0.6'}
29%|  | 500/1728 [21:07<51:05, 2.50s/it]{'loss': '0.1055', 'grad_norm': '0.1232', 'learning_rate': '1.422e-05', 'epoch': '0.6'}
33%|  | 576/1728 [24:19<48:09, 2.51s/it]

0%|  | 0/36 [00:00<?, ?it/s]
6%|  | 2/36 [00:01<00:23, 1.45it/s]
8%|  | 3/36 [00:02<00:31, 1.05it/s]
11%|  | 4/36 [00:04<00:35, 1.10s/it]
14%|  | 5/36 [00:05<00:36, 1.19s/it]
17%|  | 6/36 [00:06<00:37, 1.24s/it]
19%|  | 7/36 [00:08<00:37, 1.28s/it]
22%|  | 8/36 [00:09<00:36, 1.29s/it]
25%|  | 9/36 [00:10<00:35, 1.32s/it]
28%|  | 10/36 [00:12<00:34, 1.34s/it]
31%|  | 11/36 [00:13<00:33, 1.36s/it]
33%|  | 12/36 [00:14<00:32, 1.36s/it]
36%|  | 13/36 [00:16<00:31, 1.36s/it]
39%|  | 14/36 [00:17<00:30, 1.37s/it]
```

```
Train: 9215 Val: 1152 Test: 1152
类别数: 3
Loading weights: 100%|██████████| 199/199 [00:00<00:00, 628.65it/s]
[transformers] BertForSequenceClassification LOAD REPORT from: bert-base-uncased
Key | Status |
-----+-----+
cls.predictions.bias | UNEXPECTED |
cls.seq_relationship.bias | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
cls.predictions.transform.dense.bias | UNEXPECTED |
cls.seq_relationship.weight | UNEXPECTED |
cls.predictions.transform.dense.weight | UNEXPECTED |
classifier.weight | MISSING |
classifier.bias | MISSING |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

开始训练 BERT ...
6%|███████| 100/1728 [04:20<1:07:37, 2.49s/it]{'loss': '0.3531', 'grad_norm': '3.3', 'learning_rate': '1.885e-05', 'epoch': '0.1'}
12%|███████| 200/1728 [08:28<1:02:34, 2.46s/it]{'loss': '0.1296', 'grad_norm': '0.5548', 'learning_rate': '1.77e-05', 'epoch': '0.2'}
17%|███████| 300/1728 [12:40<59:44, 2.51s/it]{'loss': '0.09421', 'grad_norm': '16', 'learning_rate': '1.654e-05', 'epoch': '0.5206'}
```

```
92%|██████████| 33/36 [00:44<00:04, 1.39s/it]
94%|██████████| 34/36 [00:46<00:02, 1.39s/it]
97%|██████████| 35/36 [00:47<00:01, 1.40s/it]

100%|██████████| 1728/1728 [1:19:24<00:00, 2.53s/it]
100%|██████████| 36/36 [00:49<00:00, 1.40s/it]
{'eval_loss': '0.1058', 'eval_accuracy': '0.9766', 'eval_macro_f1': '0.9512', 'eval_runtime': '0.0001'}

Writing model shards: 0%|███████| 0/1 [00:00<?, ?it/s]
Writing model shards: 100%|██████████| 1/1 [00:00<00:00, 2.69it/s]
{'train_runtime': '4766', 'train_samples_per_second': '5.801', 'train_steps_per_second': '0.363', 'train_loss': '0.0717', 'epoch': '3'}
100%|██████████| 1728/1728 [1:19:25<00:00, 2.53s/it][transformers] There were missing keys in the checkpoint model loaded: ['bert.embeddings.LayerNorm.beta', 'bert.embeddings.LayerNorm.weight']
[transformers] There were unexpected keys in the checkpoint model loaded: ['bert.embeddings.LayerNorm.beta', 'bert.embeddings.LayerNorm.weight']
100%|██████████| 1728/1728 [1:19:25<00:00, 2.76s/it]

在 Test Set 上评价 ...
100%|██████████| 36/36 [00:53<00:00, 1.50s/it]

BERT Test Accuracy: 0.9826
BERT Test Macro-F1: 0.9625

已保存 task3_results.csv

进程已结束，退出代码为 0
```

结果：

| 模型                | Test Accuracy | Test Macro-F1 |
|-------------------|---------------|---------------|
| BERT-base-uncased | 0.9826        | 0.9625        |

### 3. 结果分析

1. BERT 在 NYT Test Set 上取得了 0.9826 的 Accuracy 和 0.9625 的 Macro-F1，表现稳定。
2. 与 Task 2 的三种词向量方法相比，BERT 略优于 Word2Vec on AG News 和 Word2Vec on NYT，与 GloVe 接近。
3. 与 Task 1 的 Binary BoW 相比，BERT 略低。这说明在 NYT 这个数据集上，词袋模型已经非常强，而 BERT 在 max\_length=64、3 epochs 的限制下，未能充分发挥其优势。
4. BERT 的训练 loss 从第 1 个 epoch 的 0.35 左右下降到第 3 个 epoch 的 0.07 左右，说明模型收敛良好。
5. 第 1 个 epoch 验证集 Accuracy 为 0.9644，最终测试集 Accuracy 为 0.9826，说明模型没有明显过拟合。

## 八、三种方法总体比较

将 Task 1、Task 2、Task 3 的结果汇总如下：

| 方法                  | Test Accuracy | Test Macro-F1 |
|---------------------|---------------|---------------|
| Binary BoW          | <b>0.9913</b> | <b>0.9797</b> |
| Word Frequency      | 0.9896        | 0.9771        |
| GloVe (pre-trained) | 0.9852        | 0.9646        |
| Word2Vec on AG News | 0.9792        | 0.9485        |
| Word2Vec on NYT     | 0.9826        | 0.9556        |
| BERT-base-uncased   | 0.9826        | 0.9625        |

## 综合分析

1. **词袋模型在本数据集上表现最好。** Binary BoW 和 Word Frequency 的 Test Accuracy 均超过 0.98，其中 Binary BoW 达到 0.9913。这说明 NYT 数据集的类别区分度较强，关键词是否出现已经能提供足够的信息。
2. **词向量平均表示略低于词袋模型。** GloVe、Word2Vec on AG News、Word2Vec on NYT 的 Test Accuracy 在 0.98 左右，Macro-F1 在 0.95 左右。平均词向量会丢失词序和局部信息，因此性能略低。
3. **BERT 表现稳定**，Test Accuracy 为 0.9826，Macro-F1 为 0.9625。在 max\_length=64、3 epochs 的限制下，BERT 未能超过词袋模型，但整体性能仍然较好。
4. **不同表示方法的适用场景不同：**

词袋模型适合关键词区分度高的任务而词向量平均适合需要语义泛化的任务；BERT 适合需要深层语义理解的任务，但需要更充足的训练资源和更长的序列长度。

## 九、结论

本次实验完成了三种文本表示方法在 NYT 文本分类任务上的实现与比较：

1. Task 1 使用 Binary BoW 和 Word Frequency，配合 Logistic Regression，取得了最高的 Test Accuracy (0.9913) 和 Macro-F1 (0.9797)。
2. Task 2 使用 GloVe、Word2Vec on AG News、Word2Vec on NYT 三种词向量表示，Test Accuracy 在 0.9792—0.9852 之间，Macro-F1 在 0.9485—0.9646 之间。
3. Task 3 使用 BERT-base-uncased 微调，Test Accuracy 为 0.9826，Macro-F1 为 0.9625。

实验表明，不同文本表示方法各有优劣。在 NYT 这个类别区分度较强的数据集上，传统词袋模型反而取得了最好的效果；词向量平均表示和 BERT 也表现良好，但受限于表示方式和训练条件，未能超过词袋模型。